DAT 471 – Computational Techniques for Large-Scale Data

Module 3

Group 6

Name	Programme	Email
Gabriela Castillo Avila	MPHPC	avilag@student.chalmers.se
Mahnoor Ali Cheema	MPDSC	mahnoora@student.chalmers.se

04 May 2026

Problem 1 - Star Control II

Part a - Operations the Programmer needs to define

1- Mapper Operation

Input to the Mapper A single record/line from planets.csv containing:

- Star - Greek letter or "Prime"
- Constellation - constellation name or star name (if Star = Prime)
- Mineral value (RU) - Integer

Output of the Mapper A key-value pair:

```
(star_system_name, mineral_value)
```

Mapper Pseudocode

```
function MAP(record):
star = record["Star"]
constellation = record["Constellation"]
mineral = int(record["Mineral value (RU)"])

if star == "Prime":
    system_name = constellation
else:
    system_name = star + " " + constellation

EMIT(system_name, mineral)
```

2- Reducer Operation

Input to the reducer

- Key : star system name (for example "Alpha Centauri").
- Values: list of all mineral values emitted for that star system.

Example Input to Reducer:

```
Key:"Alpha Centauri"
Values: [120, 450, 3000, 50, ....]
```

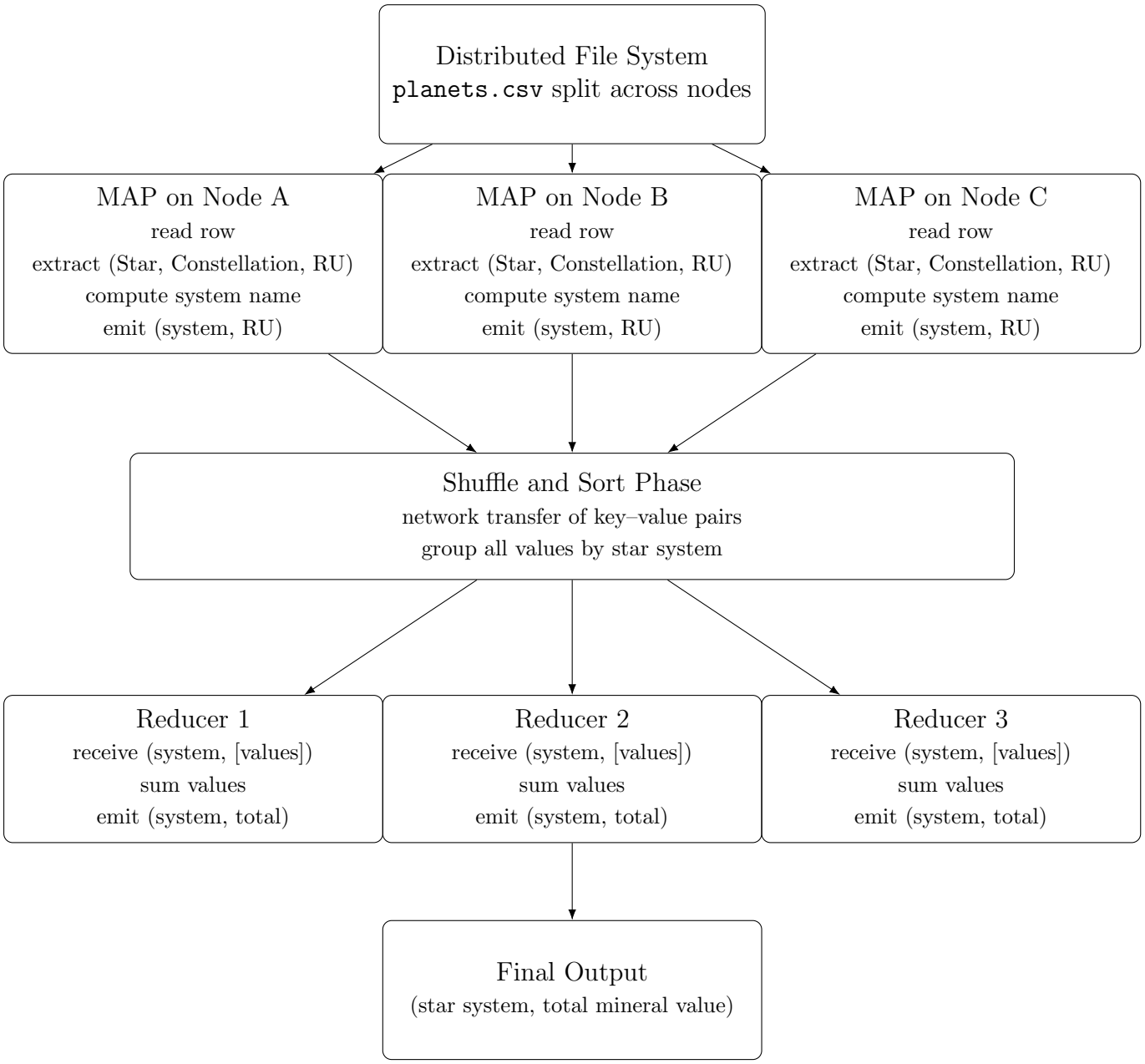
Output of the reducer A single key-value pair:

```
(star_system_name, total_mineral_value)
```

Reducer Pseudocode

```
function REDUCE(system_name, values):
total = SUM(values)
EMIT(system_name, total)
```

Part b - Data flow diagram for the MapReduce Program



Part c - Implementation of Program using MRJob

The map,reducer function which are main part of the program are implemented as follows:

```
def mapper(self, _, line):
   # Parse CSV line
   row = next(csv.reader([line]))

   # Skip header
   if row[0] == "Constellation":
       return

   constellation = row[0].strip()
```

```
    star = row[1].strip()
    mineral_str = row[5].strip()  # Mineral value (RU)

    # Build star system name (same logic as Problem 2)
    if star == "Prime":
        system_name = constellation
    else:
        system_name = f"{star} {constellation}"

    # Convert mineral value
    try:
        mineral_value = int(mineral_str)
    except ValueError:
        return

    yield system_name, mineral_value

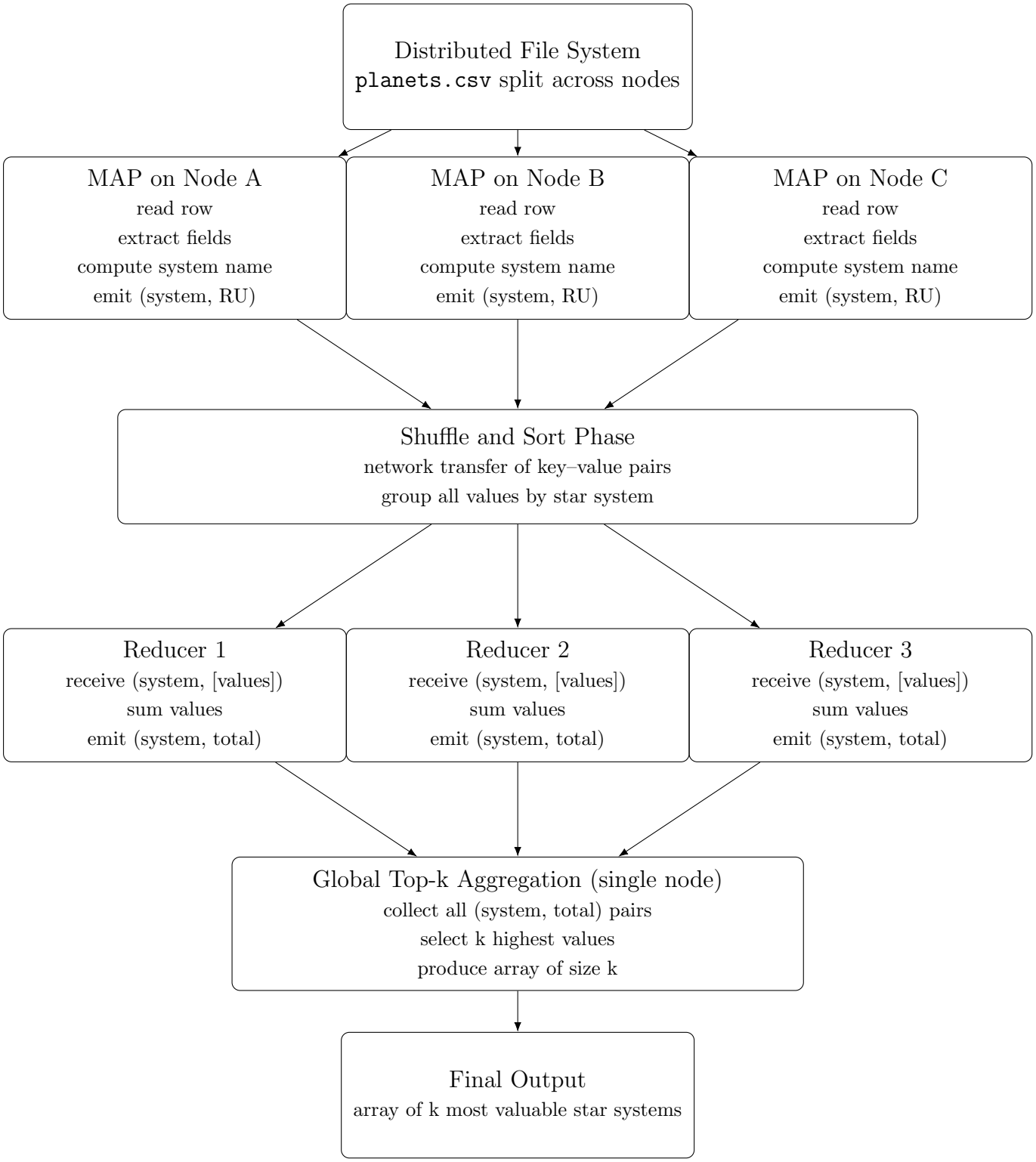
def reducer(self, system_name, mineral_values):
    yield system_name, sum(mineral_values)
```

Star System	**Total Mineral Values (RU)**
Capella	1248
Alpha Cancri	2082
Gamma Sagittae	2899
Beta Lyrae	2431
Alpha Geminorum	4257

Table 1: Total mineral values for selected star systems

Problem 2

Part a - Modified Data flow Diagram for MapReduce Program



Part b - Top 10 most valuable Star System

In addition to the implementation for Problem 1, following functions are added:

```
def mapper_prepare_topk(self, system, total):
    # Send all results to one reducer
    yield None, (system, total)

def reducer_topk(self, _, system_total_pairs):
    k = self.options.k
    heap = []

    # Maintain a min-heap of size k
    for system, total in system_total_pairs:
        if len(heap) < k:
            heapq.heappush(heap, (total, system))
        else:
            heapq.heappushpop(heap, (total, system))

    # Sort descending before output
    topk_sorted = sorted(heap, reverse=True)

    for total, system in topk_sorted:
        yield system, total
```

Rank	Star System	Total Mineral Value (RU)
1	Beta Scorpii	12680
2	Delta Tauri	12080
3	Alpha Ceti	11739
4	Alpha Centauri	11476
5	Delta Aurigae	11005
6	Beta Cephei	10447
7	Zeeman	10421
8	Zeta Vulpeculae	9404
9	Beta Normae	9153
10	Delta Brahe	9100

Table 2: Top 10 most valuable star systems by total mineral value.

Problem 3

Part a

MapReduce is used to process large datasets, such as the Twitter dataset, in parallel.

The algorithm must compute:

1. Find the ID of the person who follows the most users.
2. Find how many users that person follows.
3. calculate Average number of users followed.
4. Find how many accounts that follow zero users.

```
mapper(key, line):
   user_id , follow =  split input line at :
      if follows is empty
        id_following_count = 0
      else
           id_following_count = number of ids user follows
   results stats, (user_id, id_following_count,
       id_following_count==0)


   Local mac user_id and its following count
   sum the local followed acounts of that id
   count local users
   count the local pf the users that follow zero users

   yield stats, (local_id_most_following, local_following_max_count,
   local_total_following_all_users, local_total_number_of_users,
   local_users_zero_following)



reducer(stats, lines)
   find global following_max_count
   find global total_following_all_users
   find global total_number_of_users
   find global users_zero_following

   average = total_following_all_users / total_number_of_users

   yield:
      most followed id: id_most_following
      Most followers: following_max_count
      Average followers: average
      count no followers: users_zero_following
```

Figure 1 shows the flowchart of the data.

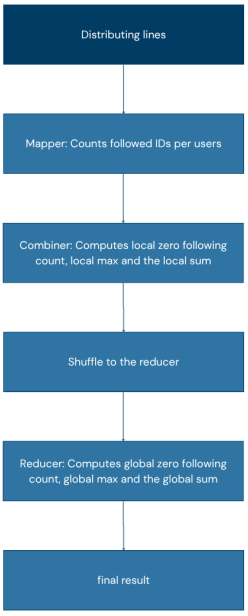


Figure 1: Flow of data

Part b

The following is the implementation of the pseudo code from problem 3a:
Mapper function:

```
def mapper(self,key, value):
    user_id, follows = value.split(':', 1)
    follows = follows.strip()

    if follows == "":
        id_following_count = 0

    else:
        id_following_count = len(follows.split(','))

    yield "stats", (user_id, id_following_count, id_following_count == 0)
```

Combiner function:

```
  def combiner(self, key, values):
    id_most_following = None
    following_max_count = -1
    following_all_users = 0
    number_users = 0
    count_zero_following = 0

    for value in values:
        user_id = value[0]
        following_count = value[1]
```

```
        following_zero = value[2]

        if following_count > following_max_count:
            id_most_following = user_id
            following_max_count = following_count

        following_all_users += following_count
        number_users += 1

        if following_zero:
            count_zero_following += 1

        yield "stats", (id_most_following, following_max_count, following_all_users,
```

Reduce function:

```
    def reducer(self,key, values):
        #combiner

        id_most_following = None
        following_max_count = -1
        total_following_all_users = 0
        total_number_users = 0
        count_zero_following = 0

        for value in values:
            local_user_id = value[0]
            following_count = value[1]
            local_following_all_users = value[2]
            local_number_users = value[3]
            local_following_zero = value[4]

            if following_count > following_max_count:
                id_most_following = local_user_id
                following_max_count = following_count

            total_following_all_users += local_following_all_users
            total_number_users += local_number_users
            count_zero_following += local_following_zero


        average_following = total_following_all_users / total_number_users

        yield ("most followed id", id_most_following)
        yield ("most followed", following_max_count)
        yield ("average followed", average_following)
        yield ("count follows no-one", count_zero_following)
```

Part c

The program was run using the following command:

```
time python3 assignment3_problem3.py -r local --num-cores 1
/data/.../datasets/twitter/twitter-2010_10M.txt
```

Running the program with one core resulted in the following output:

```
"most followed id"      "15131310"
"most followed"         119880
"average followed"      8.3638526
"count follows no-one"  3754459


real    0m37.977s
user    0m36.429s
sys     0m3.125s
```

Number of cores	Runtime	Speedup
1	37.977s	1
2	30.603s	1.24
4	16.273s	2.33
8	11.411s	3.33
16	10.450s	3.63
32	10.150s	3.74

Table 3: Speedup for different number of cores

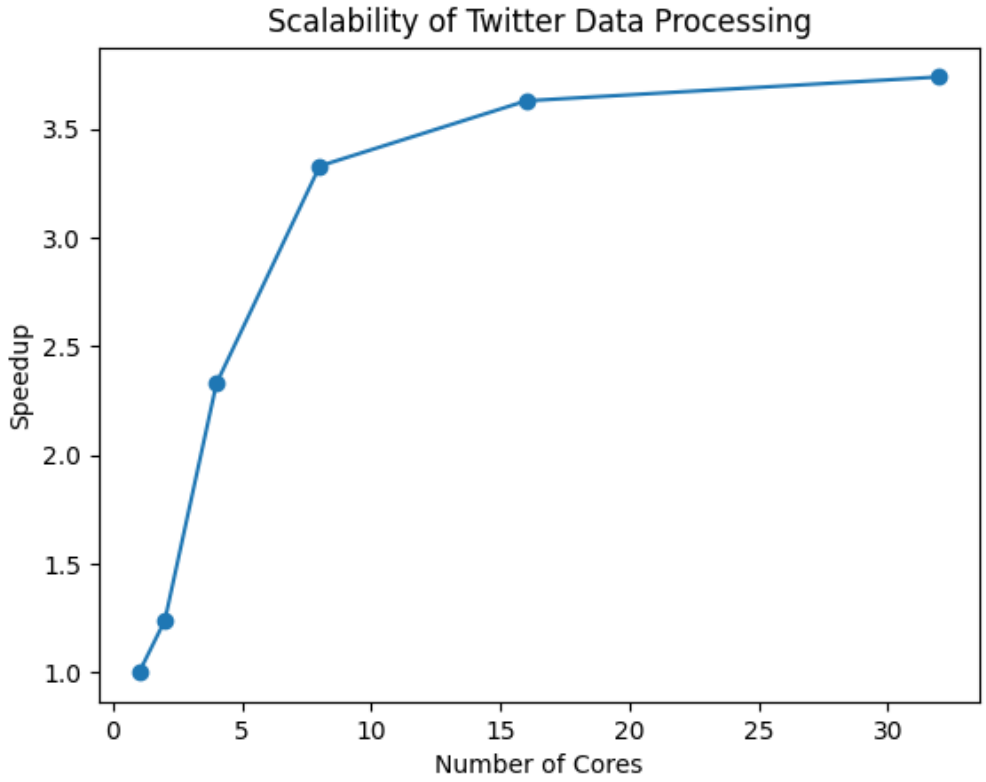


Figure 2: Flow of data

Part d

As the figure 2 in previous part shows, the curve starts to flatten out between at 16 cores. The core number of choice was therefore *32*, to keep the core number high, while maintaining good performance without wasting resources.
The output, of using the full dataset, with 32 cores, is the following:

```
"most followed id"        "813286"
"most followed"           770155
"average followed"        35.25297882010159
"count follows no-one"    5963082
```

Problem 4

Part a

The algorithm must compute:

1. Find the ID of the person who has most followers.
2. How many followers.
3. calculate Average number of users followers.
4. Find how many accounts that have zero followers.

Following the hint from the assignment, the implementation used more than one step job

```
mapper(key, line):
    user_id , follows =  split input line at :
       yield (user_id, 0)

       if follows is not empty:
           for follow in follows:
               yield (followed_id, 1)

reduce_step1(user_id, followers):
   followers_count = sum(followers)

   yield stats, (user_id, followers_count)

 combine(stats, values)
       local_id_most_followers = None
       local_followers_max_count = -1
       local_followers_all_users = 0
       local_number_users = 0
       local_count_zero_followers = 0

       for all user and their followers_count in the values
           if followers_count > local_follower_max
               local_id_most_followers = user
               local_followers_max_count = followers_count

           local_followers_all_users += follower_count
           local_number_users += 1

           if follower_count == 0:
               local_count_zero_followers += 1

       yield stats, (local_id_most_followers, local_followers_max_count,
       local_followers_all_users, local_number_users,
           local_count_zero_followers)


   reduce_step2(stats, values):
       find global follower_max_count
       find global total_followers_all_users
       find global total_number_of_users
       find global users_zero_followers

       average = total_followers_all_users / total_number_of_users

       yield:
           most followers id: id_most_followers
           most followers: followers_max_count
```

```
Average followers: average
count no followers: users_zero_followers
```

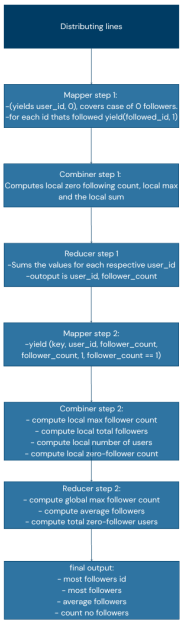


Figure 3: Flow of data part 4

Part b

The following is the implementation of the pseudo code from problem 4a:
Using mrjob.step

```
def steps(self):
    # first stepcomputes user_id and follower_count
    return [
        MRStep(mapper=self.mapper_followers,
        combiner=self.combiner_follower_count,
        reducer=self.reducer_follower_count),

        MRStep(mapper=self.mapper_stats,
        combiner=self.combiner_stats,
        reducer=self.reducer_stats)
        ]
```

Using multiple steps part 1:
mapper_followers step 1 function

```
def mapper_followers(self,key, value):
    user_id, follows = value.split(':', 1)
    follows = follows.strip()
```

```
        yield user_id, 0

        if follows != "":
           for followed_user in follows.split():
                yield followed_user, 1
```

combiner_follower_count step 1 function

```
    def combiner_follower_count(self, user_id, values):
        follower_count = sum(values)
        yield user_id, follower_count
```

reducer_follower_count step 1 function

```
    def reducer_follower_count(self,user_id, values):
        follower_count = sum(values)
        yield user_id, follower_count
```

Part 2 of the steps: *mapper_stats function*

```
    def mapper_stats(self, user_id, follower_count):
        yield "stats",
        (user_id,
        follower_count,
        follower_count,
        1,
        follower_count == 0)
```

combiner_stats function

```
    def combiner_stats(self, key, values):
        id_most_followers = None
        followers_max_count = -1
        followers_all_users = 0
        number_users = 0
        count_zero_followers = 0

        for value in values:
            user_id = value[0]
            local_follower_count = value[1]
            local_followers = value[2]
            local_users = value[3]
            local_followers_zero = value[4]

            if local_follower_count > followers_max_count:
                id_most_followers = user_id
                followers_max_count = local_follower_count

            followers_all_users += local_followers
            number_users += local_users
            count_zero_followers += local_followers_zero
```

```
        yield "stats",
        (id_most_followers,
        followers_max_count,
        followers_all_users,
        'number_users,
        count_zero_followers)
```

reducer_stats function

```
    def reducer_stats(self, key, values):
        id_most_followers = None
        followers_max_count = -1
        total_followers_all_users = 0
        total_number_users = 0
        count_zero_followers = 0

        for value in values:
            user_id = value[0]
            local_follower_count_max = value[1]
            local_followers = value[2]
            local_users = value[3]
            local_followers_zero = value[4]
            if local_follower_count_max > followers_max_count:
                id_most_followers = user_id
                followers_max_count = local_follower_count_max

            total_followers_all_users += local_followers
            total_number_users += local_users
            count_zero_followers += local_followers_zero

        average_followers = total_followers_all_users / total_number_users

        yield "most followers id", id_most_followers
        yield "most followers", followers_max_count
        yield "average followers", average_followers
        yield "count no followers", count_zero_followers
```

Part c

The program was run using the following command:

```
time python3 assignment3_problem4.py -r local --num-cores 1
/data/.../datasets/twitter/twitter-2010_10M.txt
```

Running the program with one core resulted in the following output:

```
"most followers id"     "19757371"
"most followers"        443107
```

```
"average followers"     8.3638526
"count no followers"    2485440


real    4m50.164s
user    5m37.649s
sys     0m13.812s
```

Number of cores	Runtime	Speedup
1	4m 50.164s	1
2	2m 40.758s	1.80
4	1m 41.781	2.85
8	1m 11.536s	4.06
16	1m 2.809s	4.62
32	1m 7.608s	4.29

Table 4: Speedup for different number of cores counting followers

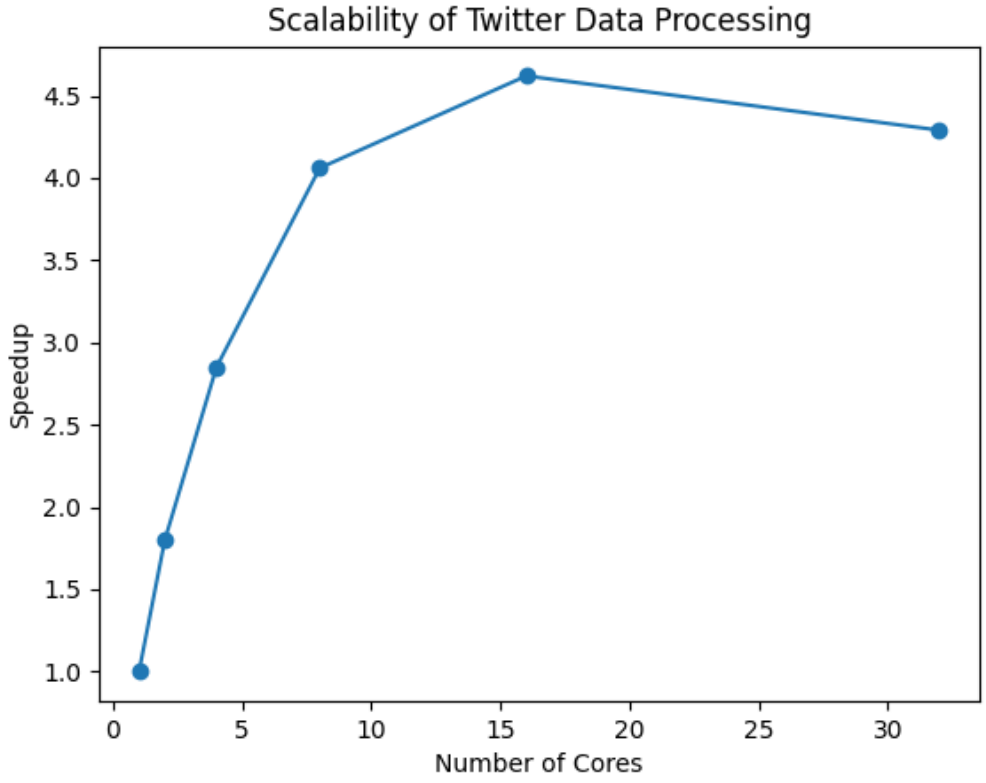


Figure 4: Flow of data

Figure 4 shows the speedup results found in table 4. The plot demonstrates a performance improvement as the number of cores increases, up until 16 cores. Beyond 16 cores the curve starts declining, indicating a diminish in performance gain from additional parallelization.

Part d

As figure 4 indicates a loss in performance from increasing cores beyond 16 in this program, 16 cores was chosen as an adequate amount to perform this task.
The following was the output of running the full set with 16 cores:

```
"most followers id"     "19058681"
"most followers"        2997469
"average followers"     35.25297882010159
"count no followers"    1548949


real    11m0.977s
user    113m46.230s
sys     3m38.671s
```